# Project 4:- Maven Selenium Project – MyMavenSeleniumApp02

## Objective

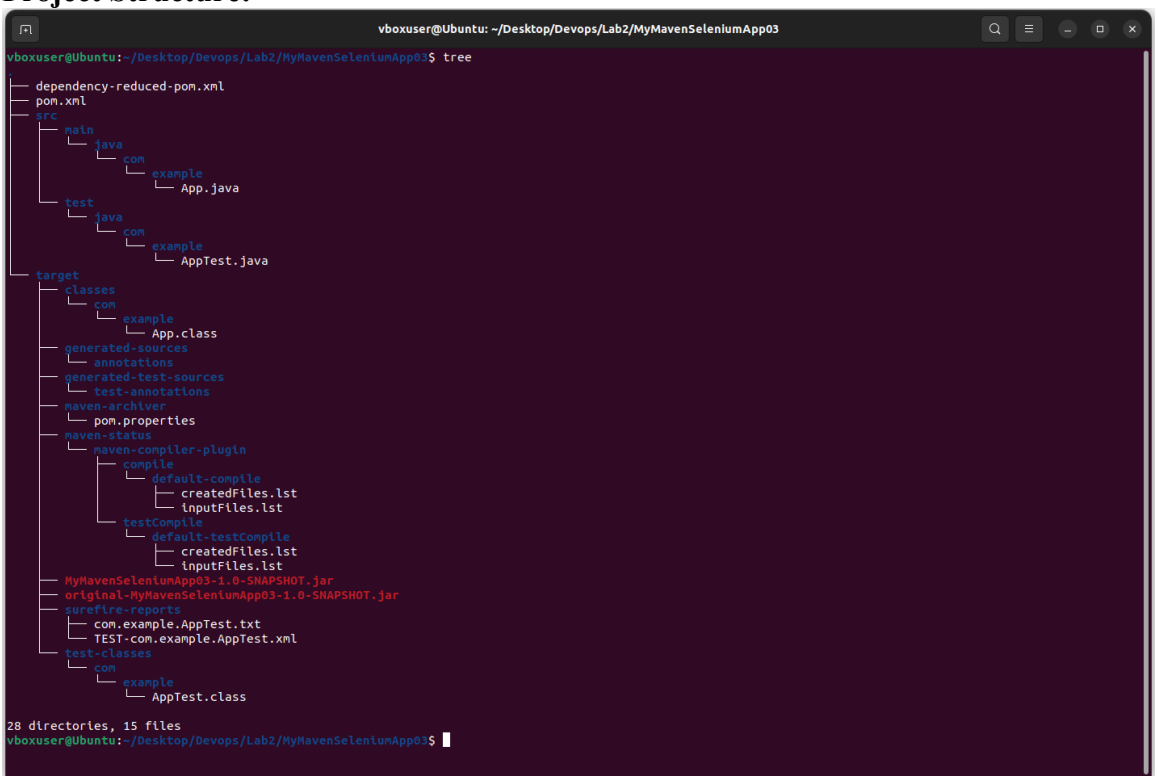
The goal of this project is to use Maven and Selenium WebDriver to write a Java program that opens a browser, finds specific elements on a webpage, and performs a series of automated interactions without any human input.

## Step 1: Create a Maven Project Using Terminal

Open a terminal window and enter the following command to scaffold the Maven project:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp02
  -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

## Project Structure:




## Step 2: Add Selenium Dependency in pom.xml

In the pom.xml file, locate the <dependencies> section and insert the entry below:

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.29.0</version>
</dependency>
```

## Step 3: Configure Maven Build Plugins

Include the Maven compiler plugin in the <build> section to specify which Java version to use:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
              <source>1.8</source>
              <target>1.8</target>
            </configuration>
          </plugin>
        </plugins>
      </build>
```

### Step 4: Inspect Elements in the Website

Launch the browser and visit the target site. Right-click on any element you plan to automate and choose Inspect from the context menu. Record the relevant attributes like id, class, or name — these will be referenced in the Java code.

### Step 5: Create App.java

Go to the path below and write the automation logic in App.java:

```
src/main/java/com/example/App.java
```

```java
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class App {
    public static void main(String[] args) throws InterruptedException {

        WebDriver driver = new ChromeDriver();
        Thread.sleep(3000);

        driver.get("http://practicetestautomation.com/practice-test-login/");
        Thread.sleep(3000);

        driver.manage().window().maximize();
        Thread.sleep(3000);

        driver.findElement(By.id("username")).sendKeys("student");
        Thread.sleep(3000);

        driver.findElement(By.id("password")).sendKeys("Password123");
        Thread.sleep(3000);

        driver.findElement(By.id("submit")).click();
        Thread.sleep(3000);

        driver.quit();
    }
}
```

### Step 6: Build the Maven Project

Use the following command to clean any previous build and recompile the project:

```
mvn clean compile
```

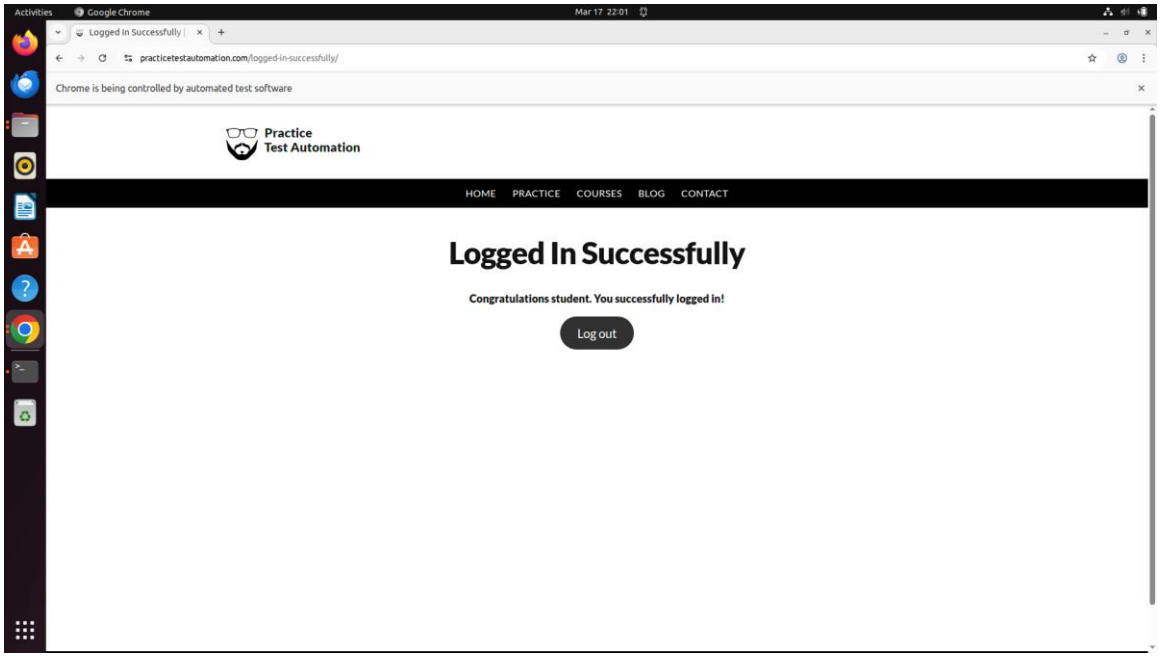
### Step 7: Run the Application

Start the application by running:

```
mvn exec:java
```

### Output

The program runs as expected, completing all the browser interactions defined in the script.




## Conclusion

Through this project, we successfully set up a Maven-Selenium environment and automated a series of browser actions end-to-end. The entire workflow ran without requiring any manual steps.

## Jenkins Pipeline for MyMavenSeleniumApp02

**Step 1: Open the Jenkins Dashboard** in order to create a new item.

**Step 2: Click on New Item** in order to create a New Item. Give the name as "MyMavenSeleniumApp02" and Item type as Pipeline, then click Create.

**Step 3: Configure the settings** by defining the Definition as "Pipeline Script from SCM" and SCM as "Git". Paste the Repository URL of the project from GitHub and choose Credentials as "GitHub Credentials" which was created earlier.

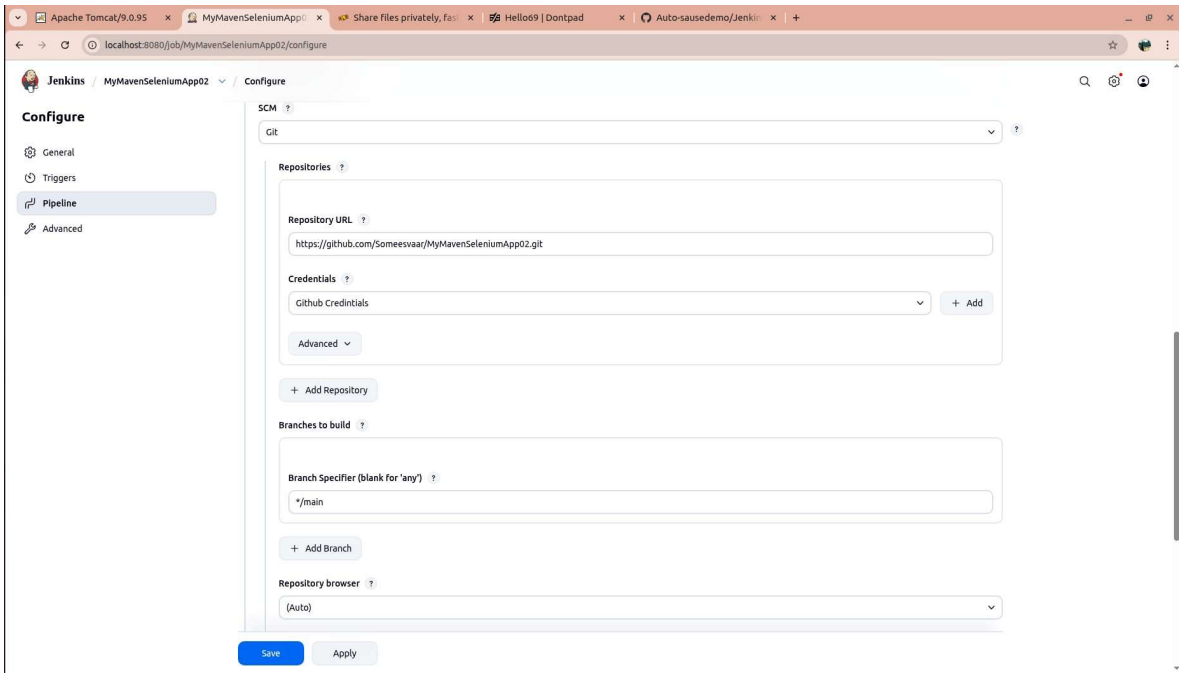



*Fig B-1: Jenkins Pipeline configuration – Repository URL for MyMavenSeleniumApp02.git with GitHub Credentials*

**Step 4: In Terminal,** go to the directory created for MyMavenSeleniumApp02 and create a new file called "Jenkinsfile".

**Step 5: Paste the following code inside the Jenkinsfile:**

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url:
'https://github.com/Someesvaar/MyMavenSeleniumApp02.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }
```

```
        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp02.jar'
            }
        }



    }
     post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
       }
    }
}
```

And add the following to App.java
options.addArguments("--headless=new");  // modern headless mode
options.addArguments("--no-sandbox");  // required in Jenkins/Linux
options.addArguments("--disable-dev-shm-usage"); // prevents crashes
options.addArguments("--disable-gpu");

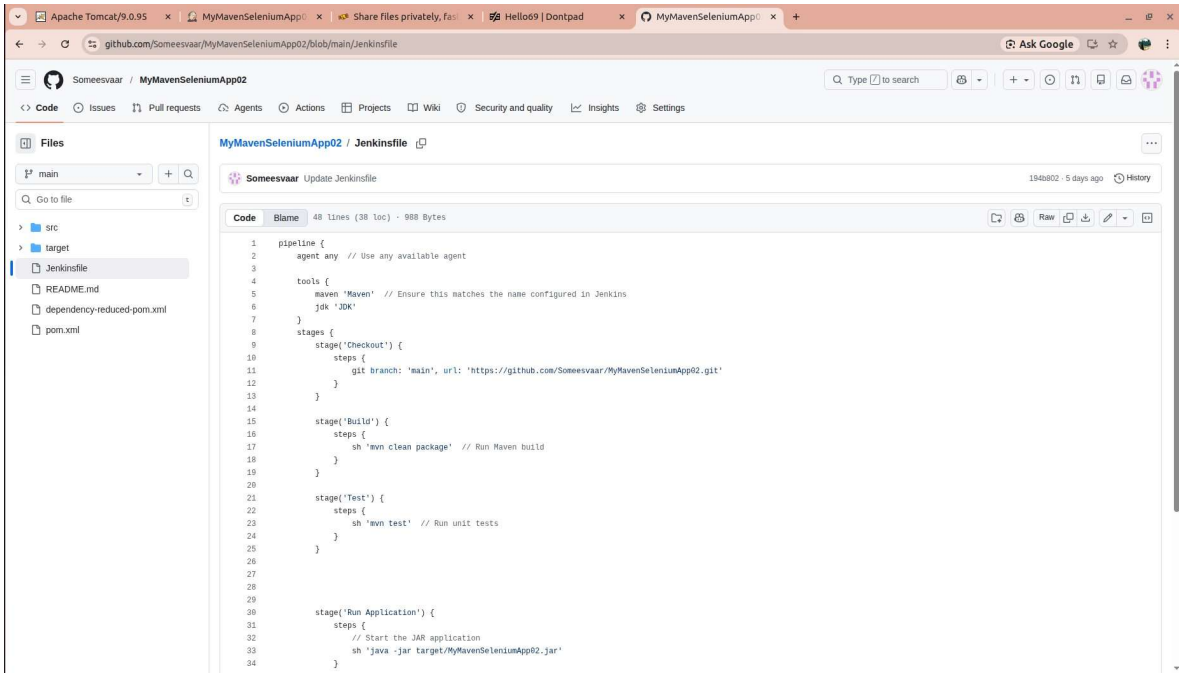



*Fig B-2: Jenkinsfile committed to the MyMavenSeleniumApp02 GitHub repository*

**Step 6: Update pom.xml** to change the versions of source and target from 1.7 to 1.8.

**Step 7: Push the changes and commit** by pushing the "Jenkinsfile" to the respective repository on GitHub.

**Step 8: Go to Jenkins and Build the Pipeline,** then check the Stage View to confirm successful execution.

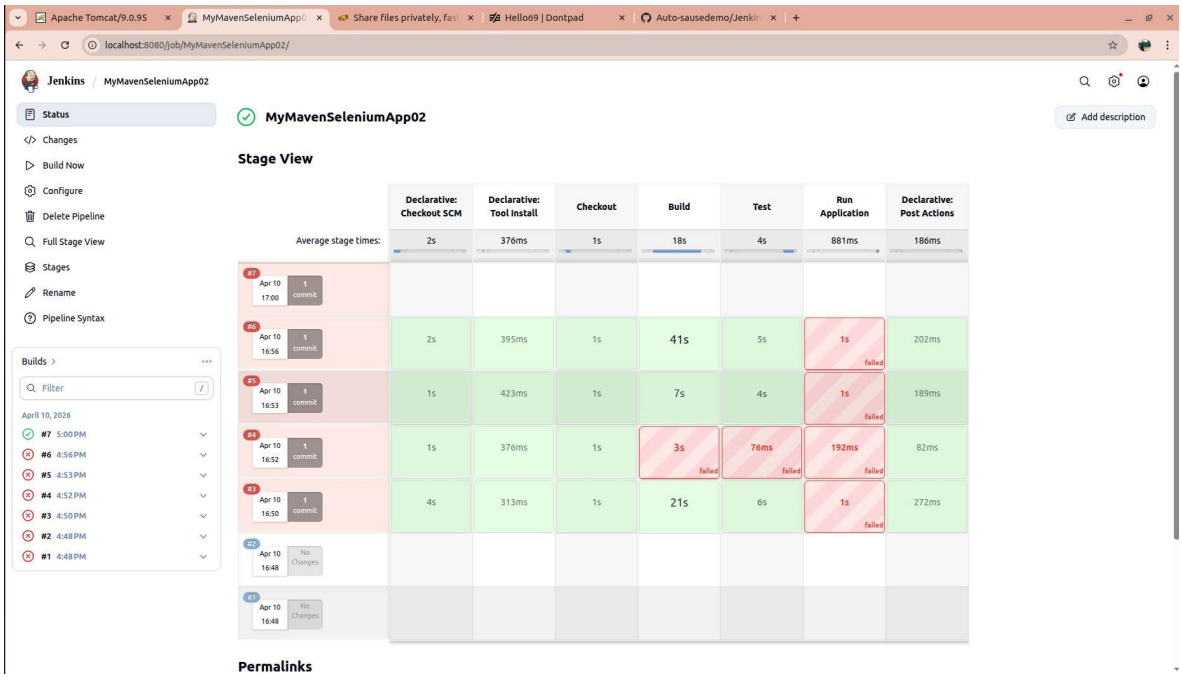



*Fig B-3: Jenkins Stage View showing build history for MyMavenSeleniumApp02 (Build #7 successful)*